

Artec 3D

Senior Front-End

Kérdésről kérdésre tudástár · React · Renderelés · SSR ·
Webteljesítmény

TARTALOM

1. React Hookok
2. A JavaScript event loop
3. React vs Next.js SEO szempontból
4. Az SSR fő előnye
5. ~1M elemű lista renderelése
6. Felesleges re-renderek megelőzése
7. Mi vált ki re-rendert
8. Hogyan dönti el a React, mit frissítsen (VDOM & reconciliation)
9. Régebbi React app migrálása React 18-ra
10. Kódminőségi eszközök
11. Mit jelent a REST
12. Web Vitals / Core Web Vitals

Mik azok a React Hookok?

React alapok

Olyan függvények, amelyekkel React funkciókat — state, életciklus, context — használhatsz **függvénykomponensekben**, class írása nélkül. A React 16.8-ban jelentek meg.

Érdemes megnevezni a fő hookokat:

- `useState` — lokális komponens-state.
- `useEffect` — mellékhatások (feliratkozások, adatlekérés, kézi DOM-műveletek), renderelés után futnak.
- `useContext` — context olvasása consumer wrapper nélkül.
- `useRef` — rendek között megőrzött, módosítható érték, ami nem vált ki re-rendert; illetve DOM-refek.
- `useMemo` / `useCallback` — számított érték / függvényidentitás memoizálása.
- `useReducer` — state reducer alapon, összetettebb átmenetekhez.

HOOKOK SZABÁLYAI

Csak a **legfelső szinten** hívd őket (soha feltételben, ciklusban vagy beágyazott függvényben), és kizárólag React függvénykomponensből vagy custom hookból. Ez teszi lehetővé, hogy a React a hívási sorrend alapján párosítsa mindegyik hookot a tárolt state-jével.

A custom hookok (egy `useX` nevű függvény, ami más hookokat hív) a stateful logika kiszervezésének és újrahasznosításának módja.

Hogyan működik a JavaScript event loop?

JS futásidő

A JavaScript **egy szálon**, egyetlen call stackkel fut. Az event loop az a mechanizmus, ami lehetővé teszi, hogy az aszinkron munkát ne blokkolja ez a szál.

Az összetevők:

- **Call stack** — itt fut a szinkron kód, egyszerre egy frame.
- **Web/host API-k** — a timer, fetch, DOM-események a motoron kívül futnak (böngésző/Node), és a végén visszaadnak egy callbacket.
- **Task queue (macrotask)** — `setTimeout`, I/O, UI-események.
- **Microtask queue** — a feloldott Promise-callbackek (`.then`, `await` folytatások), `queueMicrotask`, `MutationObserver`.

A loop: amikor a call stack üres, előbb **az összes** microtaskot lefuttatja, majd **egy** macrotaskot vesz elő, aztán megint kiüríti a microtaskokat — és ez ismétlődik. Tehát a microtaskok mindig a következő timer-callback előtt futnak.

A KLASSZIKUS BIZONYÍTÉK

A `setTimeout(..., 0)` később logol, mint egy `Promise.resolve().then(...)`, pedig mindkettő „azonnalinak” tűnik — a promise microtask, a timeout macrotask.

SEO szempontból a sima React vagy a Next.js előnyösebb?

SSR / Next.js

A Next.js az előnyösebb. Egy sima React app (kliensoldali SPA, pl. Vite/CRA) egy majdnem üres HTML-vázlat küld, és a böngészőben JavaScripttel építi fel az oldalt. A crawlerek, amelyek nem futtatják le teljesen a JS-t — vagy csak késve —, keveset vagy semmit sem látnak a tartalomról.

A Next.js a HTML-t a **szerveren** (SSR) vagy **build időben** (SSG) rendereli, így a crawler már az első válaszban teljes markuptól kap — valós tartalmat, meta tageket, címekeket.

Miért segít a rangsorolásban:

- A tartalom megvan a kliensoldali JS futtatása nélkül is.
- Oldalankénti `<title>` / meta / Open Graph a metadata API-n keresztül.
- A gyorsabb first paint javítja a Core Web Vitals-t, ami beleszámít a rangsorolásba.

Árnyalat, amit érdemes megemlíteni: a Google tud JS-t renderelni, tehát az SPA-k nem láthatatlanok — de lassabb, kevésbé megbízható, és rosszabb a nem-Google crawlereknek és a közösségi preview botoknak. A szerveroldalon renderelt HTML egyszerűen a biztonságosabb alapértelmezés az indexelendő tartalomhoz.

Mi az SSR fő előnye?

SSR / Next.js

Az oldal már az **első válaszban értelmes HTML-ként** érkezik, nem üres vázként, amit a böngészőnek kell feltöltenie. Ez két nagy előnyt ad:

- **Gyorsabb érzékelt betöltés / first contentful paint** — a felhasználó valós tartalmat lát, mielőtt a JS-bundle letöltődik és lefut.
- **Indexelhetőség és link-preview** — a crawlerek és közösségi botok azonnal teljes tartalmat kapnak.

Ezután a **hydration** hozzáköti a React eseménykezelőit a szerveroldalon renderelt markuphoz, hogy interaktívvá tegye.

TRADE-OFF, AMIT ÉRDEMES ELISMERNI

Az SSR szerverköltséggel és kérésenkénti latenciával (TTFB) jár, és az appnak nem-böngésző környezetben is futnia kell. Ott a legjobb, ahol a tartalom és a SEO számít; egy erősen interaktív, login mögötti belső dashboard sokkal kevesebbet nyer belőle.

Hogyan renderelnél egy görgethető listát ~1 millió elemmel?

Teljesítmény

Virtualizáció (windowing). Sosem teszel egymillió node-ot a DOM-ba. Csak a viewportban látható néhány sort rendereled (plusz egy kis overscan buffert), és görgetéskor lecseréled a tartalmukat.

Hogyan működik:

- Egy magas spacer elem adja a görgetősávnak a helyes teljes magasságot.
- Görgetéskor a scroll offset ÷ sormagasság alapján kiszámolod, melyik indextartomány látszik.
- Csak azokat a sorokat rendereled, abszolút pozicionálva az offsetjükre.

A gyakorlatban inkább könyvtárat használsz, mint kézzel megírod: **TanStack Virtual** (headless, és rajta van az Artec stackjén), vagy `react-window` / `react-virtualized`.

Kiegészítő technikák, amiket érdemes megemlíteni:

- **Lapozás / infinite scroll** — le se kérj egymillió sort; darabokban tölts (jól passzol a TanStack Query-hez).
- Stabil `key`-ek és memoizált sorkomponensek, hogy a görgetés ne akadozzon.
- Változó sormagassághoz mérő/dinamikus virtualizer kell.

EGY MONDATBAN

A DOM a szűk keresztmetszet, nem az adat — tartsd a mountolt node-ok számát állandón, függetlenül a lista hosszától.

Hogyan előzhetők meg a felesleges re-renderek?

React renderelés

Kezdd az alapértelmezéssel: amikor egy komponens renderel, a React újrarendereli az egész alfáját. A legtöbb „felesleges” render új prop/referencia-identitásból vagy túl magasan tartott state-ből fakad.

Eszközök, nagyjából hatókör szerint:

- **React.memo** — kihagyja a komponens re-renderelését, ha a propjai sekélyen (shallow) egyenlők a legutóbbival.
- **useMemo** — cache-eli a drága számított értéket a renderek között.
- **useCallback** — stabilan tartja egy függvény identitását, hogy a memoizált gyerekek ne lássanak „új” propot.
- **Stabil key -ek** — a helyes key-ek elkerülik a listaelemek remountolását.
- **State lejjebb / colocation** — tartsd a state-et minél lokálisabban, hogy kevesebb komponens rendereljen újra.
- **Context szétbontása / slice-ok** — egyetlen nagy context minden változásra minden fogyasztóját újrarendereli; bontsd szét, vagy használj selector-alapú store-t (Redux Toolkit, Zustand).
- **Children propként átadva** — a **children** -en át adott tartalom nem jön létre újra, amikor a szülő re-renderel.

EZT MONDD

Ne szórd reflexből mindenhová a **memo** / **useMemo** -t — mérj a React DevTools Profilerrel, aztán a forró útvonalakat memoizáld. A korai memoizálásnak megvan a maga költsége és zaja. (Érdemes megjegyezni: a React 19 React Compiler-e ebből sokat automatizál.)

Mi vált ki komponens- / UI-re-renderert?

React renderelés

Négy dolog:

- **State-változás** — egy `useState` / `useReducer` setter meghívása (olyan értékkel, ami `Object.is` szerint eltér).
- **Új propok** — a szülő újrenderelt és propokat adott le.
- **Szülő re-renderje** — a szülő renderelése alaptól újrendereli a gyerekeit, akkor is, ha a propjaik nem változtak (kivéve, ha memoizáltak).
- **Context-változás** — egy fogyasztott context-érték frissült.

Ami viszont *nem* trigger: egy `ref` módosítása, vagy egy objektum/tömb helyben (in-place) mutálása setter hívása nélkül. A React a **referenciát** hasonlítja, így egy helyben mutálás, ami ugyanazt a referenciát tartja, nem vált ki re-renderert — pontosan ezért fontosak az immutábilis frissítések.

Hogyan dönti el a React, mit frissítsen state-változás után?

React renderelés · VDOM

A **virtuális DOM-on és a reconciliationön** keresztül.

- A renderelés egy új **virtuális DOM** fát állít elő — egy könnyű, JS-alapú leírását a UI-nak.
- A React **diffeli** ezt az új fát az előzőhöz képest (reconciliation).
- Kiszámolja a valós DOM-műveletek minimális halmazát, és csak azokat commitolja. Az érintetlen node-okat békén hagyja.

A diffelés két heurisztikára épül: a **különböző típusú** elemeket teljesen kicseréli (alfa újraépül), a **key**-ek pedig azonosítják, mely gyerekek ugyanazok a renderek között, így az elemek átrendezhetők ahelyett, hogy megsemmisülnének és újra létrejönnének. A rossz key-ek (pl. tömbindex egy átrendeződő listán) gyakori forrásai a rejtett bugoknak.

EGY MONDATBAN

A virtuális DOM olcsóvá teszi a „mi változott” kérdést; a közvetlen DOM-manipuláció a drága rész, ezért a React ezt minimalizálja.

Ha a modern belsőkre rákérdeznek: a React 16 óta a reconciler („Fiber”) fel tudja darabolni a munkát egységekre, és prioritizálni/megszakítani tudja — ez a concurrent funkciók alapja.

Hogyan migrálnál egy régebbi React appot React 18-ra?

Migráció

Fogalmazd meg **kicsi, biztonságos, szállítható lépésekként** — pontosan ezt a fegyelmet keresi ez a pozíció.

- **Emeld a függőségeket** — `react` és `react-dom` 18-ra, plusz minden könyvtár, aminek kompatibilis major kell (routing, state, tesztelés).
- **Válts az új root API-ra** — a `ReactDOM.render(...)` helyett `createRoot(container).render(...)`. Ez kapcsol be a React 18 viselkedésbe és a concurrent funkciókba.
- **Számíts az automatic batchingre** — a 18 a state-frissítéseket promise-okban, timeoutokban és natív handlerekben is batcheli, nem csak a React eseményekben. Általában előny; néha felszínre hoz olyan kódot, ami a régi időzítésre épült.
- **Tesztelj `StrictMode` alatt** — dev módban szándékosan duplán hívja az effecteket/rendereket, hogy kibukjanak az unsafe mellékhatások. Javítsd az effecteket, amelyek nem idempotensek vagy nincs cleanupjuk.
- **Migrálj inkrementálisan, támaszkodj a típusellenőrzőre + tesztekre** — frissíts, futtasd a tesztsort, javítsd a warningokat, szállíts; ne írd át funkciókat közben.
- **Az új API-kat később vedd be, tudatosan** — `useTransition`, `useDeferredValue`, Suspense adatokhoz, `useId` — ha már stabilan futsz 18-on.

EZT MONDD

„Előbb érd el, hogy a 18 változtatás nélkül fusson (root API + zöld tesztek), aztán a concurrent funkciókat külön, reviewolható változtatásokként vedd be.”
Jegyezd meg, hogy ez pont az Artec CRM-jére illik, ami már React 18-on van, és a routingot TanStack Routerre migrálja.

Milyen eszközök/ellenőrzések garantálják a kódminőséget?

Eszközök / CI

Rétegezd őket, hogy a problémák egyre korábban kiderüljenek:

- **TypeScript** — statikus típusellenőrzés (`tsc --noEmit`), a bugok egész osztályait fogja meg futásidő előtt.
- **ESLint** — bugokat fog és konvenciókat kényszerít; a `react-hooks` plugin jelzi a Hookok-szabályai és az effect-függőségi hibákat.
- **Prettier** — formázás, hogy a review a lényegről szóljon, ne a stílusról.
- **Tesztek** — unit/komponens (Vitest/Jest + React Testing Library), plusz E2E (Playwright/Cypress) a kritikus folyamatokra.
- **Vizuális regresszió** — Storybook + Chromatic a nem szándékos UI-változások elkapására (rajta van az Artec UI stackjén).
- **Pre-commit hookok** — Husky + lint-staged futtatja a lintet/formázást a módosított fájlokon, mielőtt a commit landol.
- **CI** — típusellenőrzés, lint, tesztek és build minden PR-en, hogy semmi ne merge-elődjön pirosan; kötelező review.

KERETEZÉS

A lokális hookok gyors visszajelzést adnak; a CI a kikényszerített kapu. Ugyanaz az ellenőrzés, két helyen.

Mit jelent a REST?

API-k

REpresentational **S**tate **T**ransfer — egy architekturális stílus hálózati API-khoz, jellemzően HTTP fölött.

Érdemes hozzátenni egy tartalmas mondatot, hogy ne csak a betűszó legyen:

- **Erőforrások**, amiket URL-ek azonosítanak (`/users/42`), és HTTP-metódusokkal kezelsz: `GET` , `POST` , `PUT` / `PATCH` , `DELETE` .
- **Állapotmentes (stateless)** — minden kérés magával viszi, ami kell; a szerver nem tart kliens-sessiont a hívások között.
- Értelmes **státuszkódok** (2xx/4xx/5xx) és reprezentációk, általában JSON.

Opcionális szembeállítás: a GraphQL-lel szemben (egy endpoint, kliens által meghatározott alak) — a REST erőforrás- és igeorientált, több endpointtal.

Mik azok a Web Vitals / Core Web Vitals?

Webteljesítmény

A Google szabványosított metrikái a valós felhasználói élményre. A **Core Web Vitals** a három kiemelt metrika, mindegyik más dimenziót mér — betöltés, interaktivitás, vizuális stabilitás:

- **LCP — Largest Contentful Paint** (betöltés): idő, amíg a legnagyobb látható elem megjelenik. Jó: $\leq 2,5s$.
- **INP — Interaction to Next Paint** (interaktivitás): a responzivitás az összes interakción át; 2024-ben leváltotta a FID-et. Jó: $\leq 200ms$.
- **CLS — Cumulative Layout Shift** (vizuális stabilitás): mennyire ugrál a tartalom váratlanul. Jó: $\leq 0,1$.

Kiegészítő (nem core) vitalok, amiket megnevezhetsz: **FCP** (First Contentful Paint) és **TTFB** (Time To First Byte).

MIÉRT RELEVÁNS ITT

Mérhetők, beleszámítanak a rangsorolásba, és az SSR/virtualizáció/render-optimalizáció (a korábbi kérdések) pont az a kar, ami mozgatja őket — LCP a szerveroldali rendereléssel, INP a kevesebb main-thread munkával, CLS a képek/aszinkron tartalom helyének fenntartásával.